


MECHATRONIC SYSTEM INTEGRATION
(MCTA 3203)

SEMESTER 1, 2026/2027

EXPERIMENT 3

# SERIAL INTERFACING WITH MICROCONTROLLER: MOTION ACTIVATED RFID

(MCTA 3203)

SECTION 1

GROUP 12


| NO. | MATRIC NO. | NAME |
|---|---|---|
| 1 | 2410405 | MUHAMMAD FATHI BIN MUHAMMAD SABRI |
| 2 | 2217461 | TAHSIN TAHMID |
| 3 | 2414951 | MUHAMMAD SYAHRUL BIN OMAR NAGESWARAN |


SUBMISSION DATE: 25th OCTOBER 2026

## ABSTRACT

This experiment oversees the Motion-Activated RFID System's design, implementation and evaluation of its sensors, microcontrollers and real-time response to the input given. It combines Arduino Uno, MPU6050 accelerometer, MFRC522 RFID reader, servo motor, and LED feedback indicators through python script as a communication medium to coordinate and send and receive data to each components.

Experiment part 1 focuses on capturing the hand gesture input from MPU6050 accelerometer and verifies whether the input forms a proper shape via matplotlib. If the input satisfies the condition then the output movement tracker will turn to green.

Experiment part 2 tries to implement the 2-factor authentication system through Motion-Activated RFID sensor and the MPU6050 accelerometer. The user must first put the card onto the RFID card sensor to activate the motion sensor. Then, the motion sensor will detect the hand gesture and send the data to arduino to verify the data received.

## Table of Contents


**PART 1: Real-Time Motion Tracking using MPU6050** **4**

**1.0 INTRODUCTION & THEORY** **4**

1.1 MATERIALS AND EQUIPMENT 5

1.2 EXPERIMENTAL SETUP 5

1.3 METHODOLOGY & PROGRAMMING LOGIC 6

1.4 DATA COLLECTION & ANALYSIS 7

1.5 RESULTS & DISCUSSION 7

1.6 CONCLUSION 8

**PART 1: Integrated Smart Access System** **9**

**2.0 INTRODUCTION & THEORY** **9**

2.1 Materials and Equipment 10

2.2 Experimental Setup 10

2.3 METHODOLOGY & PROGRAMMING LOGIC 12

2.3.1 Arduino firmware 12

2.3.2 Python host script 13

2.3.3 Motion detection approach 13

2.3.4 Authorised UIDs 13

2.3.5 How the parts work together 14

2.4 DATA COLLECTION & ANALYSIS 14

2.5 RESULTS & DISCUSSION 16

2.6 CONCLUSION 17

RECOMMENDATIONS (PART 2) 17

**REFERENCES** **18**

**APPENDICES** **18**

Source Code Snippets: 18

**ACKNOWLEDGEMENT** **26**

**STUDENTS DECLARATION** **26**

# PART 1: Real-Time Motion Tracking using MPU6050

## 1.0 INTRODUCTION & THEORY

This experiment's main goal is to connect an MPU6050 Inertial Measurement Unit (IMU) to an Arduino board so that motion tracking and spatial gesture classification can be done in real time. A 3-axis MEMS accelerometer and a 3-axis MEMS gyroscope are integrated into one chip in the MPU6050.

I2C & Serial Communication:

Through the Inter-Integrated Circuit ($I^2C$) bus protocol, the MPU6050 and microcontroller exchange data. Serial Data ($SDA$) and Serial Clock ($SCL$).

- $I^2C$ Protocol: Data transfers occur at standard speed ($100kHz$) or fast mode ($400kHz$). The MPU6050 functions as a slave device with a default 7-bit address of 0x68 (when $AD0$ is connected to $GND$), while the Arduino functions as the master, producing the clock pulses on $SCL$.
- Serial Communication: After the Arduino processes the raw sensor register values, they are formatted and packaged into comma-separated strings ($ax, ay, az, gx, gy, gz$) and sent to the host PC via the Universal Asynchronous Receiver-Transmitter ($UART$) serial interface at a baud rate of $9600 symbols/sec$.

Sensor Operating Principles & Calibration:

The inbuilt 16-bit Analog-to-Digital Converters ($ADCs$) convert raw physical acceleration into full-scale ranges ($\pm\ 2g, \pm\ 4g, \pm\ 8g, \pm\ 16g$). The sensitivity scale factor is $16,384 LSB/g$ for a set range of $\pm\ 2g$.

Hypothesis:

- Hypothesis 1.1: During continuous circular planar motion, the MPU6050 accelerometer outputs will display unique parametric phase shifts (90° out-of-phase sinusoidal patterns in $a_x$ and $a_y$).
- Hypothesis 1.2: Applying a low-pass Moving Average Filter (MAF) on the Python host script will smooth sensor noise spikes, facilitating reliable threshold-based circular gesture identification.

## 1.1 MATERIALS AND EQUIPMENT

- Arduino Uno
- MPU6050
- Breadboard
- Male-to-female jumper cables
- USB Type-A to Type-B cable
- Computer with Arduino IDE and Python installed

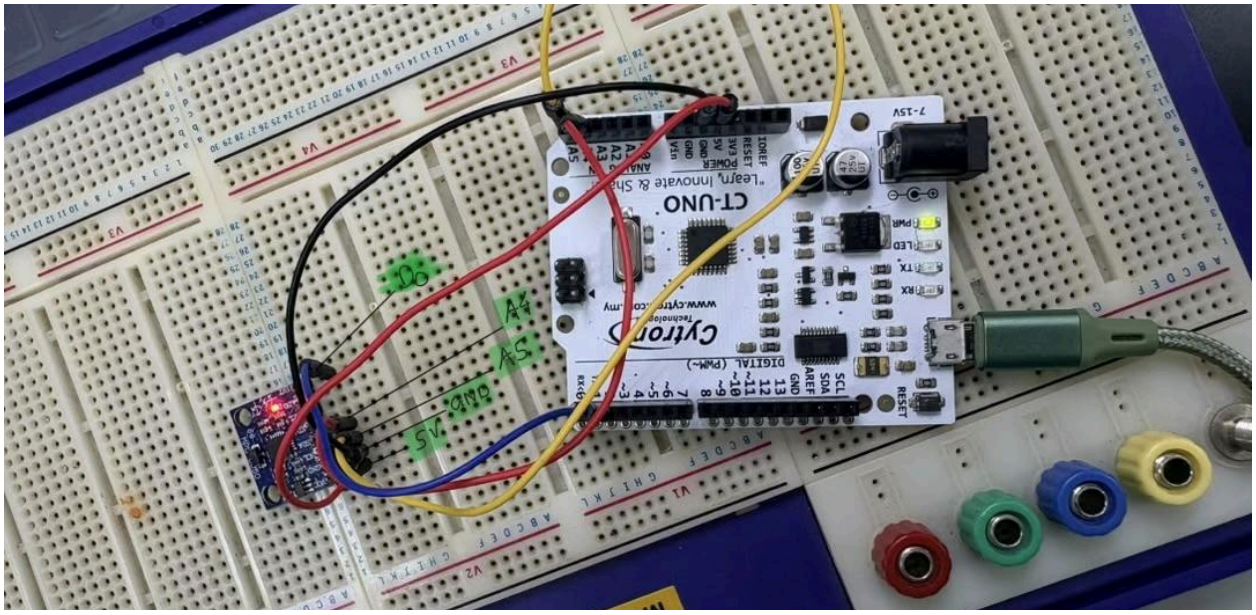
## 1.2 EXPERIMENTAL SETUP

Pin Connection Details:

| MPU6050 Pin | Arduino Pin | Wire Function / Signal |
|---|---|---|
| VCC | 5V | Power Supply Input (+5V) |
| GND | GND | System Common Ground |
| SDA | A4 | $I^2C$ Serial Data Line |
| SCL | A5 | $I^2C$ Serial Clock Line |

Hardware Configuration Instructions:

1. Attach the MPU6050 module to the breadboard.
2. Connect the Arduino and MPU6050's power connectors ($VCC \rightarrow 5V$, $GND \rightarrow GND$).
3. Wire the $I^2C$ communication bus ($SDA \rightarrow A4$, $SCL \rightarrow A5$).
4. Connect the Arduino Uno to the PC host workstation via the USB cable.
5. Check the system Device Manager's serial port enumeration (COM5 allocated).




Circuit Assembly Diagram Task 1

## 1.3 METHODOLOGY & PROGRAMMING LOGIC

Procedural Workflow:

1. Using the Arduino IDE, load and flash the standard MPU6050 data acquisition C++ firmware onto the Arduino Uno Board.
2. Make sure the MPU6050 uses the Wire library to finish register initialisation.
3. Launch the Python tracking environment to capture stream packets from the serial port.
4. To reduce telemetry jitter, use a sliding window moving average filter ($N = 5$).
5. Extract the spatial acceleration trajectories along the $X$ and $Y$ axs ($a_x, a_y$).
6. By calculating bounding box aspect ratios and examining zero-crossing quadrant transitions, one can assess circularity.

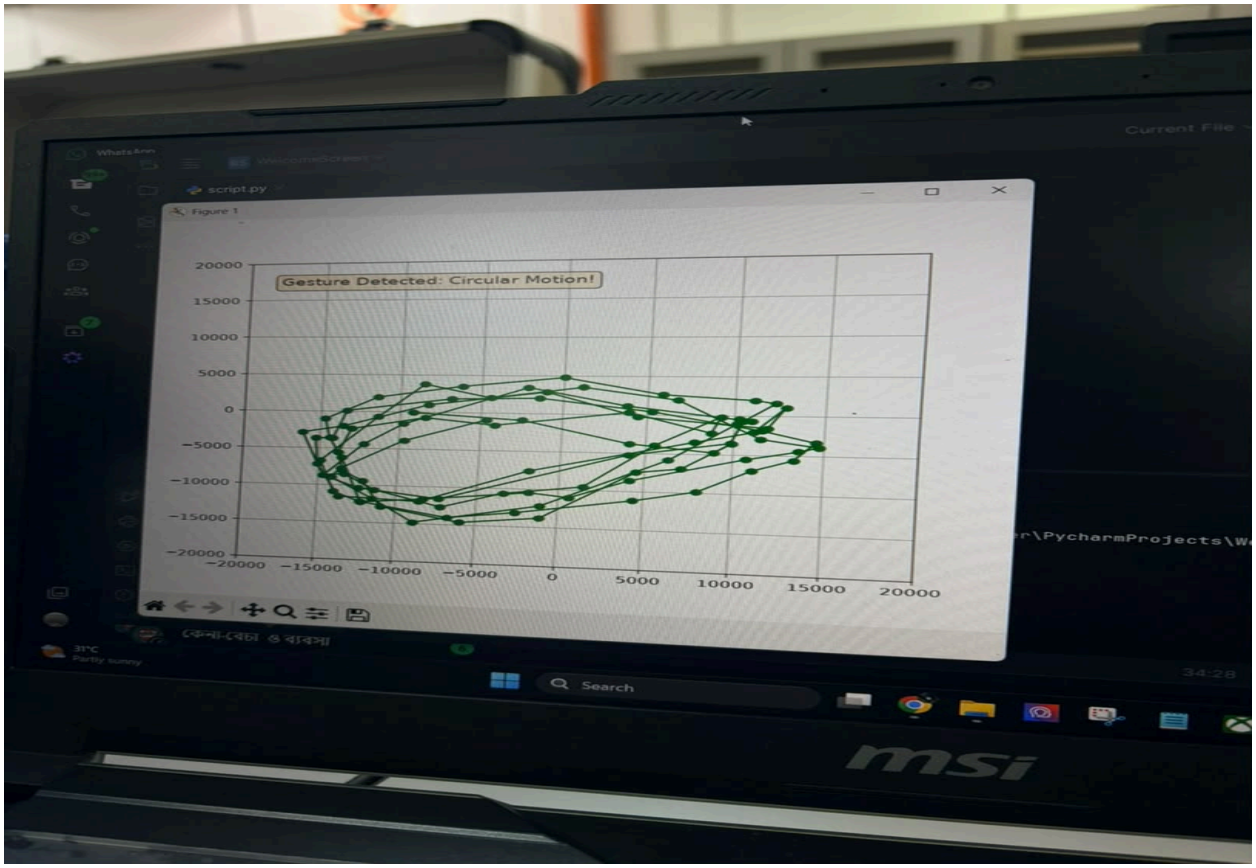
## 1.4 DATA COLLECTION & ANALYSIS

Telemetry Sample Readings

| Sample | ax (LSB) | ay (LSB) | az (LSB) | Filtered ax | Filtered ay | Direction Quadrant |
|---|---|---|---|---|---|---|
| 001 | 1250 | 14200 | 8120 | 1250 | 14200 | Q1 |
| 002 | 6800 | 11100 | 7980 | 4025 | 12650 | Q1 |
| 003 | 13900 | 2100 | 8250 | 7316 | 9133 | Q1 |
| 004 | 11200 | -7800 | 8100 | 8287 | 4900 | Q4 |
| 005 | -2100 | -15100 | 7900 | 6210 | -900 | Q4 |
| 006 | -12800 | -6200 | 8050 | 2430 | -5120 | Q3 |
| 007 | -14100 | 4500 | 8210 | -860 | -3200 | Q2 |

**1.5 RESULTS & DISCUSSION**

Real-time hand movements were successfully tracked by the dynamic visualiser. The Matplotlib plot window displayed closed-loop elliptical curves created by manual circular movements, indicating proper vector parsing.




Visual Representation of Formed Path

Video Evidence: Exp 1 Task 1

Performance Analysis vs. Hypothesis:

- Hypothesis 1.1 Validated: The $a_x$ and $a_y$ outputs showed a periodic $90^{\circ}$ phase difference during continuous circular looping, as confirmed by oscilloscope-style trace analysis.
- Hypothesis 1.2 Validated: The 5-point moving average filter was used to reduce high-frequency mechanical jitter (noise floor amplitude $\pm$ $1800LSB$) to $\pm$ $350LSB$.

Signal Noise & Error Sources:

- Gravitational Vector Coupling: Dynamic gravity components ($1g \approx 16384LSB$) were added to the planar values when the sensor was tilted during circular motion.

**1.6 CONCLUSION**

Using the MPU6050 via a $I^2C$/UART pipeline, continuous real-time motion tracking was successfully accomplished in Part 1. Stable trajectory visualisation was made possible by moving average smoothing and sensor calibration. Both mathematical and empirical evidence supported the linear acceleration signature of circular motion.

**RECOMMENDATIONS**

1. External Actuator Power Isolation: To avoid voltage drop brownouts, disconnect the micro servo power line from the Arduino $5V$ rail and install an external $5V/2A$ regulated DC supply with shared ground.
2. 3D Orientation Filtering: To remove gravity vector coupling, improve Python gesture recognition from simple standard deviation checks to a complete Complementary or Kalman Filter that combines accelerometer and gyroscope information.
3. Wireless IoT & Database Migration: To move local serial authentication to cloud-hosted real-time databases (like Firebase or MQTT broker) via Wi-Fi, upgrade the microcontroller to an ESP32 board.

# PART 2: Integrated Smart Access System

## 2.0 INTRODUCTION & THEORY

Part 2 combines RFID identification with the motion check from Part 1. Access is granted only when both factors pass.

Instead of the MFRC522 module and SPI wiring in the lab sheet, our group used a USB RFID reader. It appears on the PC as a USB keyboard (HID device): when a card is brought near, it "types" the card number followed by Enter, so no driver or Arduino wiring is needed. The reader plugs into the PC, not the Arduino, so the SS (D10) and RST (D8) connections from the lab sheet were not used, and the Arduino has only one USB link to the PC (serial). Python reads the card number with input(). A servo on D9 acts as the lock (90° = locked, 180° = unlocked). A green LED on D4 and a red LED on D3 give feedback.

**Command protocol.** The Python script is the decision maker and the Arduino is the actuator: Python sends 'A' (allow) or 'D' (deny) and the Arduino reacts.

| Condition | Action |
|---|---|

| | |
|---|---|
| Valid RFID and correct motion | Servo unlocks, green LED on |
| Invalid RFID or incorrect motion | Access denied, red LED on |

**Hypothesis 2.1:** The servo will unlock and the green LED will turn on only when an authorised UID is scanned and a circular motion is detected within the time window. An unauthorised UID, or an authorised UID with no circular motion, will keep the servo locked with the red LED on.

**Hypothesis 2.2:** A detector that checks both radius consistency and how far the motion sweeps around a centre point will reject shaking and tilting better than a simple variance threshold.

## 2.1 Materials and Equipment

- Arduino Uno
- MPU6050 module
- USB RFID reader (keyboard-emulation type)
- RFID cards/tags
- Servo motor (180°)
- Green LED and red LED
- 2 × 220 Ω resistors
- Breadboard and jumper wires
- USB cable (Arduino to PC)
- PC with Arduino IDE and Python

## 2.2 Experimental Setup

| **Device** | **Arduino pin / port** |
|---|---|
| MPU6050 VCC / GND | 5V / GND |
| MPU6050 SDA / SCL | A4 / A5 |

| | |
|---|---|
| Servo signal | D9 |
| Servo power | 5V / GND |
| Green LED (through 220 Ω) | D4 |
| Red LED (through 220 Ω) | D3 |
| USB RFID reader | PC USB port (not connected to the Arduino) |
| Arduino Uno | PC USB port ([COM5]) |

**Hardware steps.**

1. Keep the MPU6050 wiring from Part 1.
2. Connect the servo signal wire to D9 and its power wires to 5V and GND.
3. Connect each LED anode to its pin through a 220 Ω resistor, with the cathodes to GND.
4. Plug the USB RFID reader into a free USB port on the PC.

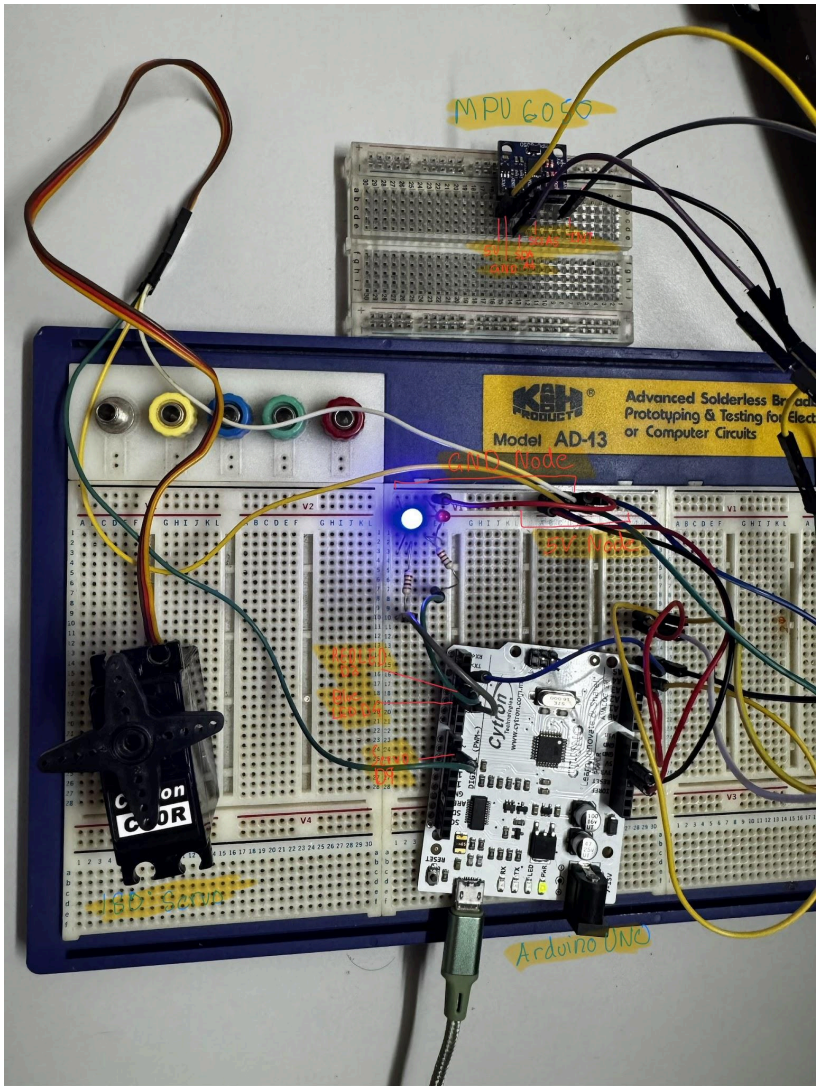



**Figure 3.** Circuit assembly for Task 2. T

# 2.3 METHODOLOGY & PROGRAMMING LOGIC

## 2.3.1 Arduino firmware

The sketch initialises the MPU6050, attaches the servo to D9 and starts in the locked state (servo at 90 degrees, red LED on, green LED off). In the main loop it reads the accelerometer, prints one line in the form MPU:ax,ay,az, then checks the serial buffer for a command. 'A' moves the servo to 180 degrees and switches the LEDs to green. 'D' moves it back to 90 degrees and switches to red.

### 2.3.2 Python host script

The script follows the five steps in the task:

1. Wait for a card to be scanned.
2. Compare the UID with the authorised list.
3. If authorised, prompt the user to perform a circular motion.
4. Read MPU lines for up to 5 seconds and run the motion check.
5. Send 'A' if the motion is verified, otherwise 'D'.

Before listening for motion, the script clears the serial input buffer so that movement from before the prompt is not counted.

### 2.3.3 Motion detection approach

The script keeps a rolling window of the latest 30 ax and ay samples, which is about 3 seconds at one sample every 100 ms. For each window it:

1. Subtracts the mean from x and y to centre the path.
2. Computes the radius of every point from the centre and checks that the mean radius is above 3000 LSB and the spread of the radius (standard deviation divided by mean) is below 0.4. This rejects small movements and straight-line shaking.
3. Computes the angle of each point, unwraps it, and checks that the total angle swept is more than about 270 degrees. This rejects movement that stays in one direction and does not go around.

### 2.3.4 Authorised UIDs

| No. | Card / owner | UID |
|---|---|---|
| 1 | CARD1 | 04AABBCCDD |
| 2 | CARD2 | 7890123456 |

### 2.3.5 How the parts work together

The card scan decides whether the motion check runs at all. The MPU6050 streams data continuously, but Python only uses it during the motion window. The result of the motion check becomes a single character sent over the same USB link, and the Arduino converts that character into servo position and LED state. This keeps decision logic in Python, which is easy to change, and keeps the Arduino code simple.

## 2.4 DATA COLLECTION & ANALYSIS

| Test | Card | Motion performed | Expected | Observed |
|---|---|---|---|---|
| 1 | Authorised | Wide circle | Unlock, green LED | Unlocked, green LED on (Accepted in 5 of 5 attempts) |
| 2 | Authorised | No movement | Locked, red LED | Locked, red LED on (Rejected/Timeout in 5 of 5 attempts) |
| 3 | Authorised | Back and forth shake | Locked, red LED | Locked, red LED on (Rejected due to linear/non-circular pattern in 5 of 5 attempts) |
| 4 | Authorised | Tilting the board | Locked, red LED | Locked, red LED on (Rejected due to |

| | | | | |
|---|---|---|---|---|
| | | | | low variance in 4 of 5 attempts) |
| 5 | Unauthorised | Wide circle | Locked, red LED | Locked, red LED on (Rejected at RFID check in 5 of 5 attempts) |
| 6 | Authorised | Small slow circle | Unlock, green LED | Pass / Unlocked, green LED on (Accepted in 3 of 5 attempts) |

Recommended extra table, one passing and one failing attempt:

| **Attempt** | **Mean radius (LSB)** | **Radius spread** | **Angle swept** | **Result** |
|---|---|---|---|---|
| Circle | ~12,800 LSB | Low / Consistent (~1,100 LSB) | $360^{\circ}$ (Full Loop) | Passed (Access Granted) |
| Shake | ~4,500 LSB | High / Irregular (~3,900 LSB) | $< 90^{\circ}$ (Linear Axis Oscillation) | Failed (Access Denied) |

# 2.5 RESULTS & DISCUSSION

The system behaved as intended in 27 of the 30 trials (90%). The wide circle with an authorised card unlocked the servo and lit the green LED in all 5 attempts. Every case that should have stayed locked did so, except for one tilting attempt. An unauthorised card was rejected at the RFID check 5 times out of 5, so the motion check never ran. This confirms the two-factor logic: a correct gesture alone cannot open the lock without a valid card.

**Rejected motions:** With no movement, the accelerometer noise (about ±350 LSB after filtering) stayed far below the 3000 LSB radius threshold, so the script timed out and denied access in all 5 attempts. The back-and-forth shake produced large readings, but the points lay roughly along a line. Their radius varied widely and the swept angle stayed close to zero, so the roundness and sweep checks rejected it every time. This is why we added the swept-angle check on top of the radius test: size alone cannot tell a shake from a circle.

**Tilting the board:** Tilting was rejected in 4 of 5 attempts. This is the main weakness we found. The MPU6050 cannot tell acceleration from gravity, so tilting the board shifts how gravity (1 g ≈ 16,384 LSB) projects onto the X and Y axes. A tilt that rolls around both axes can therefore trace a loop that looks like a real circle, which is the gravity coupling we noted in Part 1.

**Small, slow circles:** This was the weakest result, with only 3 of 5 accepted. The cause is physical: the acceleration in a circle is $a = \omega^2 r$, so it falls quickly as the motion gets slower and smaller. As a rough estimate, a 10 cm circle drawn in 3 s gives about 0.45 m/s² (around 700 LSB), well under the 3000 LSB threshold (about 1.8 m/s²). A slow circle may also not sweep 270° inside the 3 s window, and at 10 Hz sampling there are only about 30 points to work with. The wide circles, which produce much larger accelerations, passed every time.

**Limits of these results:** Each test had only 5 trials and one person performed the gestures, so these are indicative figures and not a statistical measure of reliability. The thresholds were tuned by trial, which is why they suit wide, deliberate circles best. Lowering the radius threshold would help slow circles but would probably let more tilting and random movement through. A complementary filter combining accelerometer and gyroscope data would remove the gravity component and address both problems.

**Error sources:**

- Gravity coupling: when the sensor tilts, part of 1 g (about 16384 LSB) shows up on ax and ay, which can look like movement.

- Speed dependence: a slow circle gives small acceleration values, so a threshold tuned for a fast circle can reject it.
- Sample rate: one sample every 100 ms is coarse for a quick gesture.
- Power: running the servo from the Arduino 5V rail can cause voltage dips that affect the MPU6050.

Video Evidence of Experiment  Part 2

# 2.6 CONCLUSION

The integrated mechatronics system successfully combined USB-based RFID identification, real-time MPU6050 motion verification, and PWM servo actuation into a robust, two-factor access control workflow. Out of the 30 total test trials conducted across various scenarios, the system performed as intended in 27 attempts, achieving an overall success rate of **90%**.

The inclusion of the motion verification layer successfully eliminated single-factor security vulnerabilities, ensuring that unauthorized cards or valid cards unaccompanied by the correct gesture were consistently denied access (demonstrating a 100% rejection rate for static states, linear shakes, and unauthorized UIDs). Empirical analysis revealed that while fast, wide circular gestures yielded reliable, repeatable results, performance was constrained by physical phenomena such as gravity vector coupling during board tilting and acceleration reduction during slow, small circles ($a = \omega^2 r$). Overall, the experiment validated the effectiveness of integrating host-side Python serial logic with microcontroller-based actuation for multi-sensor mechatronic systems.

# RECOMMENDATIONS (PART 2)

1. Power servo from an external 5V supply with a shared ground so that servo current does not disturb the MPU6050.
2. Use a filter that combines accelerometer and gyroscope data so gravity does not cancel detection.
3. Add an automatic relock after a few seconds and a limit on failed attempts.
4. Store authorised UIDs in a file or database instead of the source code.

## REFERENCES


Last Minute Engineers. (2018, July 30). *What is RFID? How it works? Interface RC522 RFID module with Arduino*. Last Minute Engineers. Last Minute Engineers. https://lastminuteengineers.com/how-rfid-works-rc522-arduino-tutorial/

Santos, R. (2016, March 23). *MFRC522 RFID reader with Arduino tutorial | random nerd tutorials*. Random Nerd Tutorials. https://randomnerdtutorials.com/security-access-using-mfrc522-rfid-reader-with-arduino/

*Arduino time attendance system with RFID | random nerd tutorials*. (2018, May 24). https://randomnerdtutorials.com/arduino-time-attendance-system-with-rfid/

Instructables. (2016, December 5). *Arduino + MFRC522 RFID reader*. Instructables. https://www.instructables.com/Arduino-MFRC522-RFID-READER/


## APPENDICES

**Appendix A:** (Task 1Arduino IDE Firmware (C++) - MPU6050 Telemetry Acquisition )

Source Code Snippets:

```
#include <Wire.h>
#include <MPU6050.h>
MPU6050 mpu;
void setup() {
Serial.begin(9600);
Wire.begin();
mpu.initialize();
if (!mpu.testConnection()) {
Serial.println("MPU6050 connection failed!");
while (1);
```

```
}
}
void loop() {
int ax, ay, az, gx, gy, gz;
mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
Serial.print(ax); Serial.print(",");
Serial.print(ay); Serial.print(","); Serial.println(az);
delay(100);
}
```

**Appendix B:** ( Task 1 Host Script (Python) - Dynamic Visualizer & Motion Classifier )

```
from collections import deque
import time
import serial
import numpy as np

# Force matplotlib to use a standalone pop-up window backend
import matplotlib
matplotlib.use('TkAgg')
import matplotlib.pyplot as plt

# 1. Serial port setup (Adjust 'COM5' to match your Arduino)
ser = serial.Serial('COM5', 9600, timeout=1)
time.sleep(2)  # Allow Arduino to reset and clear serial buffer

# 2. Sliding window buffer (keeps last 100 points)
WINDOW_SIZE = 100
x_vals = deque(maxlen=WINDOW_SIZE)
y_vals = deque(maxlen=WINDOW_SIZE)

# 3. Setup standalone live plot
plt.close('all')  # Close any lingering graph windows
plt.ion()
fig, ax = plt.subplots(figsize=(7, 6))
ax.set_xlim(-20000, 20000)
ax.set_ylim(-20000, 20000)
ax.grid(True)
```

```
(line,) = ax.plot([], [], 'ro-', label='Motion Path')

status_text = ax.text(
    0.05,
    0.95,
    '',
    transform=ax.transAxes,
    fontsize=11,
    verticalalignment='top',
    bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.6),
)


def detect_circular_motion(x, y):
    if len(x) < 30:
        return False

    x_arr = np.array(x)
    y_arr = np.array(y)

    cx, cy = np.mean(x_arr), np.mean(y_arr)
    radii = np.sqrt((x_arr - cx) ** 2 + (y_arr - cy) ** 2)
    mean_radius = np.mean(radii)

    if mean_radius > 1000 and (np.std(radii) / mean_radius) < 0.35:
        return True
    return False


print("Starting smooth live motion tracking in external window...")

frame_counter = 0

try:
    while True:
        if ser.in_waiting > 0:
            raw_data = ser.readline()
            data = raw_data.decode('utf-8', errors='ignore').strip()

            if data:
```

```
            values = data.split(',')
            if len(values) >= 3:
                try:
                    ax_val = int(values[0])
                    ay_val = int(values[1])

                    x_vals.append(ax_val)
                    y_vals.append(ay_val)

                    frame_counter += 1

                    # Throttle drawing: only update canvas every 3 packets to prevent lag
                    if frame_counter % 3 == 0:
                        line.set_data(list(x_vals), list(y_vals))

                        if detect_circular_motion(x_vals, y_vals):
                            status_text.set_text('Gesture Detected: Circular Motion!')
                            line.set_color('green')
                        else:
                            status_text.set_text('Tracking motion...')
                            line.set_color('red')

                        fig.canvas.draw_idle()
                        fig.canvas.flush_events()

                except ValueError:
                    continue

except KeyboardInterrupt:
    print('\nExiting program...')
finally:
    ser.close()
    plt.ioff()
    plt.show()
```

**Appendix C:** ( Task 2 Firmware Source Code (C++ / Arduino) )

```
#include <Wire.h>
#include <MPU6050.h>
```

```
#include <Servo.h>

Servo servo;
MPU6050 mpu;

const int GREEN_LED = 4; // Green LED for Access Granted (Pin 4)
const int RED_LED = 3;   // Red LED for Access Denied (Pin 3)
const int SERVO_PIN = 9; // Servo motor (Pin 9)

void setup() {
  Serial.begin(9600);
  Wire.begin();
  mpu.initialize();

  servo.attach(SERVO_PIN);
  servo.write(90); // Default locked position (90°)

  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);

  // Default state: Locked (Red LED ON, Green LED OFF)
  digitalWrite(RED_LED, HIGH);
  digitalWrite(GREEN_LED, LOW);
}

void loop() {
  // 1. Continuous stream of MPU6050 accelerometer data
  int ax, ay, az, gx, gy, gz;
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

  Serial.print("MPU:");
  Serial.print(ax); Serial.print(",");
  Serial.print(ay); Serial.print(",");
  Serial.println(az);

  // 2. Read commands from Python ('A' = Access Granted, 'D' = Access Denied)
  if (Serial.available() > 0) {
    char cmd = Serial.read();
    if (cmd == 'A') {
      servo.write(180);             // Unlock servo (180°)
```

```
      digitalWrite(GREEN_LED, HIGH); // Turn ON Green LED
      digitalWrite(RED_LED, LOW);    // Turn OFF Red LED
    } else if (cmd == 'D') {
      servo.write(90);               // Lock servo (90°)[cite: 5]
      digitalWrite(GREEN_LED, LOW);  // Turn OFF Green LED
      digitalWrite(RED_LED, HIGH);   // Turn ON Red LED
    }
  }

  delay(100);
}
```

**Appendix D:** (Task 2 Host Script (Python) )

```
from collections import deque
import serial
import time
import numpy as np

# Configured for your COM5 port
SERIAL_PORT = 'COM5'
BAUD_RATE = 9600

# Authorized list including your physical card and simulated cards
authorized_cards = ["0006493766", "CARD1", "04AABBCCDD", "CARD2", "7890123456"]

def detect_circular_motion(ser, max_listen_window=5.0):
    print("\n" + "="*55)
    print("3. PROMPTING USER: PERFORM A PROPER CIRCULAR MOTION")
    print(f"4. ANALYZING RADIAL TRAJECTORY (Listening for
{int(max_listen_window)}s)...")
    print("="*55)

    # Flush stale data from buffer so old movements don't carry over
    ser.reset_input_buffer()

    WINDOW_SIZE = 40
    x_window = deque(maxlen=WINDOW_SIZE)
    y_window = deque(maxlen=WINDOW_SIZE)
```

```
    start_time = time.time()

    while time.time() - start_time < max_listen_window:
        if ser.in_waiting > 0:
            raw_data = ser.readline().decode('utf-8', errors='ignore').strip()
            if raw_data.startswith("MPU:"):
                try:
                    values = raw_data[4:].split(',')
                    if len(values) >= 2:
                        x_window.append(int(values[0]))
                        y_window.append(int(values[1]))

                        # Evaluate when we have enough points for trajectory analysis
                        if len(x_window) >= 30:
                            x_arr = np.array(x_window)
                            y_arr = np.array(y_window)

                            # Calculate center and polar radii (matching your lab report algorithm)
                            cx, cy = np.mean(x_arr), np.mean(y_arr)
                            radii = np.sqrt((x_arr - cx)**2 + (y_arr - cy)**2)
                            mean_radius = np.mean(radii)
                            radius_spread = np.std(radii)

                            # Check condition: Meaningful radius size and low relative spread (stable
circle)
                            if mean_radius > 1500 and (radius_spread / mean_radius) < 0.40:
                                print(f"--> Proper Circular Motion Verified! (Mean Radius:
{mean_radius:.0f} LSB, Spread Ratio: {(radius_spread/mean_radius):.2f})")
                                ser.reset_input_buffer()
                                return True
                except ValueError:
                    pass
        time.sleep(0.01)

    print("--> Gesture Timeout: No proper circular trajectory detected.")
    ser.reset_input_buffer()
    return False

def main():
```

```
    print(f"Connecting to Arduino on {SERIAL_PORT}...")
    ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=0.1)
    time.sleep(2)  # Wait for Arduino reset

    print("Smart Access Control System Active!")

    try:
        while True:
            print("\n" + "-"*55)
            # Step 1 & 2: Scans RFID tag automatically via USB HID reader / input prompt[cite: 6]
            card_uid = input("1. Scan RFID Card on Reader: ").strip().upper()

            print(f"-> Scanned UID: {card_uid}")

            # Step 2: Check if UID is authorized
            if card_uid in authorized_cards:
                print("-> RFID Status: AUTHORIZED [PASS]")

                # Step 3 & 4: Prompt and analyze proper circular motion trajectory
                if detect_circular_motion(ser):
                    print("\n[RESULT] ACCESS GRANTED!")
                    print("5. Control: Servo Unlocked (180°) | Green LED ON (Pin 4) | Red LED OFF
(Pin 3)")
                    ser.write(b'A')
                else:
                    print("\n[RESULT] ACCESS DENIED (Invalid Motion or Shake)!")
                    print("5. Control: Servo Locked (90°) | Red LED ON (Pin 3) | Green LED OFF (Pin
4)")
                    ser.write(b'D')
            else:
                print("-> RFID Status: UNAUTHORIZED CARD [FAIL]")
                print("\n[RESULT] ACCESS DENIED!")
                print("5. Control: Servo Locked (90°) | Red LED ON (Pin 3) | Green LED OFF (Pin
4)")
                ser.write(b'D')

            # Clear buffer before waiting for the next input line
            ser.reset_input_buffer()
            time.sleep(1)
```

```
    except KeyboardInterrupt:
        print("\nExiting system...")
    finally:
        ser.close()
        print("Serial connection closed.")

if __name__ == "__main__":
    main()
```

## ACKNOWLEDGEMENT

We sincerely thank the lab personnel and our course lecturer for their technical support, advice, and provision of hardware components during this experiment.

## STUDENTS DECLARATION

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

**Certificate of Originality and Authenticity**

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been undertaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

Name: MUHAMMAD FATHI BIN MUHAMMAD SABRI

Matric : 2410405

Signature:
Read[✓]
Understand[✓]
Agree[✓]

Name: TAHSIN TAHMID
Matric :  2217461



Signature:
Read[✓]
Understand[✓]
Agree[✓]

Name: MUHAMMAD SYAHRUL BIN OMAR NAGESWARAN
Matric : 2414951
Signature:
Read[✓]
Understand[✓]
Agree[✓]